

TOPICAL REVIEW

A Review of Time Series Prediction Models Based on Deep Learning

XIQIN AO¹, YUJIE GONG², AND AIXIANG HE¹¹Institute of Big Data and Artificial Intelligence, Anhui Xinhua University, Hefei, Anhui 230088, China²Sichuang Electronics Company Ltd., Hefei, Anhui 230088, China

Corresponding author: Xiqin Ao (aoxiqin@axhu.edu.cn)

This work was supported in part by the Intelligent Information Processing Research and Innovation Team under Grant 2024AH010012, in part by Anhui Provincial University Science Research Project under Grant 2024AH050610, and in part by the Institute of Intelligent Information under Grant yjs202103.

ABSTRACT Time series data, capturing the evolution of specific variables over time, underpins critical forecasting tasks across diverse domains. While traditional models struggle with the inherent complexity (nonlinearity, non-stationarity, seasonality) of real-world time series, deep learning (DL) has emerged as a transformative solution, leveraging its prowess in nonlinear modeling and automated feature extraction. This paper provides a comprehensive review of recent advancements in DL-based time series prediction models. We systematically analyze the characteristics, strengths, and limitations of prominent model families, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs) (e.g., LSTM, GRU), Transformer variants (e.g., Informer, Autoformer, iTransformer), Graph Neural Networks (GNNs), and hybrid models. On this basis, we establish a structured, task-oriented framework for model selection and cross-comparison, evaluating these models across essential dimensions such as sequence length handling, multivariate support, interpretability, computational efficiency, and real-time performance. Finally, addressing the challenges and issues faced by deep learning in time series prediction, this paper prospects future research directions, including enhancing model interpretability, optimizing model structures, and integrating multi-source data and domain knowledge.

INDEX TERMS Research review, time series prediction, deep learning.

I. INTRODUCTION

Time series data, a collection of data points arranged in chronological order, reflects the trend of specific variables changing over time [1], [2]. With the rapid development of information technology, time series data has experienced explosive growth in fields such as finance, meteorology, industry, transportation and healthcare, including examples like stock price trends, temperature and precipitation changes, equipment condition monitoring, traffic flow statistics, and forecast patient vital signs [3], [4], [5]. Accurately predicting the future trends of time series data is of great significance for decision-making, resource optimization, and risk early warning. Consequently, time series prediction has become a significant research focus in both academia and industry.

The associate editor coordinating the review of this manuscript and approving it for publication was Wei Xu¹.

Several traditional time series prediction techniques, like the ARIMA model and its variants, have demonstrated their efficacy in dealing with linear and stationary time series data and have managed to attain some degree of success. [6]. However, real-world time series data often exhibit complex characteristics such as nonlinearity, non-stationarity, and seasonality. Traditional methods show significant limitations in processing such data, as they struggle to capture deep-level patterns, leading to insufficient prediction accuracy.

Over the past few years, deep learning technology has achieved remarkable breakthroughs in areas like image recognition and natural language processing, and it has started to be utilized in time series prediction tasks [7], [8]. Deep learning models, through their multi-layer neural network architectures, can automatically learn complex features and dependencies in time series data, thereby effectively handling nonlinear and non-stationary data and significantly improving prediction accuracy and stability.

This paper first systematically discusses the problem of time series prediction, then reviews related research methods, and provides a detailed analysis and comparison of deep learning-based approaches according to different network types. Finally, the paper prospects the future development directions of time series prediction technology, including enhancing model interpretability, optimizing network structures, and integrating multi-source data. This paper aspires to offer valuable references for researchers in relevant fields and to facilitate the further advancement and application of deep learning technology in time series prediction.

II. BASIC CONCEPTS AND RELATED WORK

A. CONCEPTS AND PROBLEM DEFINITION

Time series prediction refers to the task of forecasting future values based on historical observed data using time series models. Its core lies in analyzing time series data, identifying patterns, trends, and periodic characteristics, and constructing appropriate prediction models to achieve accurate forecasts.

From a formal definition perspective, the time series prediction problem can be described as follows: Given an original time series dataset, where each input is a set of dynamic variables $X_t = \{x_{t,1}, x_{t,2}, \dots, x_{t,k}\}$ at time t , with k being the number of dynamic features. The output of the model is the predicted label value y_t . Typically, the input to the model is a fixed-size set $I = \{X_{t-m}, \dots, X_t\}$, representing continuous time series data from time $t-m$ to t ; the output is a set $O = \{y_{t+1}, \dots, y_{t+n}\}$, representing the model's predictions for n consecutive time points after time t [9]. Here, the input contains k variables, and the output consists of coherent predicted values for n timestamps.

When performing time series prediction, the first step is to collect and organize observed data with temporal characteristics. These data can come from various fields, such as sales over a period, temperature changes, stock prices, etc. From the original time series data, valuable features are extracted through feature engineering for effective use by the model. Proper selection and construction of features can significantly enhance the performance of the prediction model.

B. RELATED WORK

In recent years, with the explosive growth of deep learning models in the field of time series forecasting, a substantial body of related survey literature has emerged [3], [8], [10]. These reviews systematically outline the evolutionary trajectory of model architectures, their core principles, and their applications in specific domains. However, analysis of the existing literature reveals several critical research gaps:

(1) A lack of practical model selection guidelines. Existing surveys tend to focus either on in-depth analysis of single-category models (e.g., RNNs, Transformers) or provide enumerative descriptions of a broad range of models. Yet, they lack a systematic, application-oriented framework for model selection and comparison. Consequently,

researchers and practitioners face significant challenges in efficiently and effectively selecting appropriate models based on intrinsic model properties, data characteristics, and task requirements when tackling specific forecasting problems (e.g., short-term financial volatility prediction vs. long-term power load forecasting vs. multivariate traffic flow prediction).

(2) Insufficient cross-model comparative analysis and limited discussion on integration trends. While the strengths and weaknesses of various models (CNN, RNN, Transformer, GNN) have been discussed, there remains a notable deficiency in deep cross-comparisons of their core capabilities—such as capturing long/short-term dependencies, computational efficiency, robustness to noise/non-stationarity/seasonality, and interpretability. Furthermore, analysis of how recent model integrations and architectural innovations (e.g., CNN-LSTM, TCN-Attention, GNN-Transformer) address the inherent limitations of traditional models remains underdeveloped.

(3) Inadequate coverage of strategies for emerging challenges. As application scenarios grow increasingly complex and data scales expand, challenges like model interpretability, computational efficiency optimization, few-shot learning, online learning capabilities, and domain knowledge integration have become paramount. Existing surveys provide insufficient systematic summarization of these emerging challenges and targeted critiques of deep learning solutions designed to address them.

The core contributions of this survey are:

(1) Establishing a structured, task-oriented analysis framework for deep learning models. We not only elaborate on the core principles and evolution of mainstream models—including CNNs, RNNs (LSTM/GRU), Transformers (Informer, Autoformer, etc.), and GNNs (Section III)—but also place significant emphasis on conducting in-depth comparative analysis and applicability assessments of these models. This assessment is based on key time series characteristics (e.g., dependency length, trend/seasonality strength, inter-variable relationships, noise levels) and core task requirements (e.g., forecasting horizon, real-time constraints, interpretability needs).

(2) Providing in-depth discussions on model synergy and innovation. Beyond analyzing the merits and drawbacks of individual models, we focus on the design philosophy of hybrid models and structural innovations (e.g., TCN, SCINet) and critically examine their effectiveness in overcoming limitations inherent to traditional models (e.g., gradient vanishing in RNNs, computational complexity of Transformers).

(3) Placing significant emphasis on cutting-edge challenges and pragmatic solutions. We specifically focus on and critique the latest research advances addressing critical challenges, including model interpretability (e.g., attention weight analysis), computational efficiency optimization (e.g., sparse attention, model distillation), online/incremental learning, and the integration of multi-source data and domain knowledge.

III. TIME SERIES PREDICTION METHODS BASED ON DEEP LEARNING

A. OVERVIEW OF DEEP LEARNING

Time series prediction is a crucial technology for forecasting future trends by analyzing historical data, and it is widely applied in fields such as financial market analysis, weather forecasting, and healthcare. With the continuous advancement of data collection technologies, the scale and complexity of time series data have significantly increased. Traditional statistical methods and machine learning models face numerous challenges when dealing with large-scale, nonlinear data, such as high computational complexity and difficulties in capturing nonlinear relationships within the data [11]. Deep learning models, with their powerful nonlinear modeling capabilities and automatic feature extraction, have gradually become effective tools for addressing time series prediction problems.

Deep learning models, particularly deep neural networks, are valued for their capability to automatically extract high-level features from data through multiple layers of nonlinear transformations. Deep neural networks build representations of historical time series features through multiple nonlinear layers to uncover the internal patterns of time series data. [12]. During the learning process of neural networks, relevant historical time series information can be considered as being encoded into hidden variables, which ultimately become the output of these hidden variables and the current input. For different application scenarios, various implementations of hidden layers are adopted in deep learning, leading to the use of different neural network architectures. Through research, deep learning-based methods have been categorized systematically.

B. METHODS BASED ON CONVOLUTIONAL NEURAL NETWORKS

1) CNN

Convolutional Neural Networks (CNNs) are a type of deep learning model particularly well-suited for image recognition and processing tasks. Their core strength lies in feature extraction through convolutional layers and pooling layers. CNNs perform convolution operations on input data using sliding convolutional kernels, effectively extracting local spatial features. By employing a weight-sharing strategy—where the same convolutional kernel uses identical weight parameters across different positions of the input—CNNs significantly reduce the number of network parameters, thereby improving computational efficiency [13]. CNN consist of an input layer, convolutional layers, pooling layers, fully connected layers, and an output layer, as illustrated in Figure 1.

CNNs excel at extracting local spatial patterns through convolutional filters and pooling operations, benefiting from parameter sharing for efficiency and translation invariance. However, standard CNNs often struggle to capture long-range dependencies inherent in many time series and lack inherent mechanisms to model temporal order explicitly.

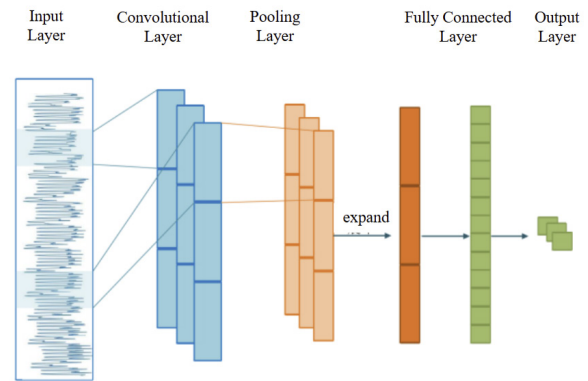


FIGURE 1. Structure of one-dimensional convolutional network.

2) TCN

The Temporal Convolutional Network (TCN) is an innovative architecture that is based on convolutional neural networks. Unlike traditional CNNs, TCN is able to capture long-term dependencies in time series data by using dilated causal convolutions and residual connections [14]. The core idea of TCN is to expand the receptive field of convolutions to capture long-range temporal dependencies while avoiding the leakage of future information.

Dilated causal convolution is the core component of TCN, combining the advantages of causal convolution and dilated convolution [15]. However, a major drawback of causal convolution is that capturing long-range dependencies requires either extremely deep networks or large filters, which significantly increases computational complexity. To address this issue, TCN introduces dilated convolution, which expands the receptive field by increasing the dilation factor (d) of the convolutional kernel. The structure of dilated causal convolution is illustrated in Figure 2.

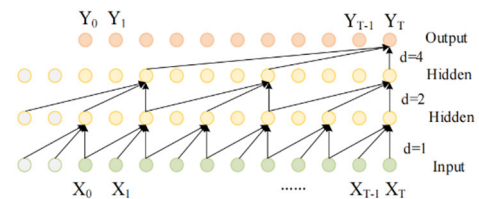


FIGURE 2. Structure of dilated causal convolution.

TCN Residual Block: As the depth of the network increases, TCN training may face issues such as vanishing or exploding gradients. To address this problem, TCN introduces residual connections [16]. Residual connections alleviate training difficulties in deep networks by directly adding the input to the output, improving model stability and convergence speed. The structure of a TCN residual block is shown in Figure 3.

In summary, TCN adheres to two principles: the input and output lengths produced by the network are the same, and no future information is leaked. These principles are achieved

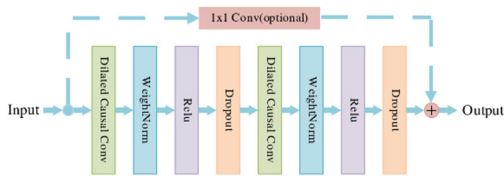


FIGURE 3. Structure of TCN residual block.

through the use of one-dimensional fully convolutional networks (1D FCN) and causal convolutions, as illustrated in the figure above. The formula for TCN can be simplified as [17]:

TCN = 1D FCN + causal convolutions (1)

3) OTHER CNN-BASED IMPROVED MODELS

In the field of time series prediction, many researchers have improved traditional Convolutional Neural Networks (CNNs) by incorporating specific application scenarios, proposing various efficient CNN variants. These improved models significantly enhance the performance of CNNs in time series prediction tasks by introducing new network structures or optimization strategies.

In 2017, Borovykh et al. [18] proposed an improved CNN model inspired by the WaveNet speech sequence generation model. This model adopts the ReLU activation function and parametric skip connections, simplifying the structure. Experiments show that this model performs exceptionally well in financial time series prediction tasks, not only simplifying the training process but also maintaining high prediction accuracy in noisy data environments. This improvement demonstrates the potential of convolutional networks in handling complex time series data.

As dataset sizes continue to grow, traditional CNNs face challenges in computational efficiency when processing large-scale data. In 2017, Dong et al. [19] proposed an improved model that combines CNN with the K-means clustering algorithm. This model employs K-means clustering to divide large datasets into smaller subsets of similar samples and subsequently trains CNN models on each of these subsets individually. This approach has shown excellent performance in prediction tasks on million-scale power load datasets, significantly improving computational efficiency and prediction accuracy.

Liu et al. [20] proposed SCINet (Sample Convolution and Interaction Network), which adopts a hierarchical convolutional network structure. SCINet uses a binary tree-like architecture to iteratively extract and aggregate features at various time resolutions, thereby learning effective representations that have improved predictability. Within each SCI-Block, the time series is split into two parts. As the depth of the binary tree grows, the network is able to extract more detailed temporal information. In this manner, SCINet is capable of capturing both short-term and long-term dependencies in time series data at the same time, which leads to a substantial enhancement in prediction performance.

However, the method relies on a fixed hierarchical network structure, which may lead to insufficient adaptability to complex or dynamically changing time series.

4) COMPARATIVE ANALYSIS OF CNN-BASED ALGORITHMS
Convolutional Neural Networks (CNNs) and their improved variants have demonstrated excellent performance in time series prediction tasks, particularly when the sample size is sufficient, as they can effectively handle short-term prediction tasks. A comparison analysis of CNN-based algorithms is summarized in Table 1.

TABLE 1. Analysis of CNN-based algorithms.

Algorithm	Strengths	Limitations
CNN	-Strong feature extraction capability	-Difficulty in processing long sequences
	-Parameter sharing reduces computational cost	-Lack of intrinsic mechanisms for handling sequence order
	-Translation invariance to data	
TCN	-Effective in handling long-term dependencies	-Poor adaptability to varying sequence lengths
	-Strong parallel computing capability	-Potential issues with vanishing or exploding gradients
WaveNet-CNN	-Excellent performance in audio processing	-High computational resource consumption
	-Hierarchical structure captures multi-scale features	-Limited adaptability to other types of data
Kmeans-CNN	-Combines data clustering with feature extraction	-Clustering results depend on initial parameters
	-Reduces data complexity and noise impact	-Increased complexity due to model fusion
SCINet	-Effective in handling seasonal and trend-based data	-Strong specificity limits application scope
	-Multi-resolution analysis capability	-Difficulty in model interpretation and understanding

CNN: Suitable for short-term time series prediction tasks, especially in scenarios with prominent local features(e.g., ECG abnormality detection).

TCN: Suitable for long-sequence prediction tasks, capable of capturing long-term dependencies(e.g.,long term forecasting of urban power load).

WaveNet-CNN: Primarily applicable to audio and time series generation tasks(e.g.,speech emotion synthesis), with limited suitability for other types of time series prediction.

Kmeans-CNN: Suitable for large-scale time series prediction tasks, effectively handling noise and complexity in data(e.g., Smart grid abnormal power consumption detection).

SCINet: Suitable for time series prediction tasks with significant seasonality and trends, capable of capturing multi-scale temporal dependencies (e.g.,holiday retail sales forecast).

Beyond the listed strengths and limitations, CNN variants involve key trade-offs:

Long-range vs. Efficiency/Complexity: TCN and SCINet address standard CNNs' limited receptive field (TCN via dilated convolutions, SCINet via hierarchical decomposition). However, this significantly increases model depth, complexity, and computational cost, making them less efficient and harder to train/tune than shallow CNNs.

Automated Features vs. Interpretability/Specificity: While powerful for local pattern extraction, the learned features are often opaque. Models like SCINet (designed for trends/seasonality) or K-means-CNN (dependent on clustering) risk poor generalization if data characteristics mismatch their design assumptions, and offer limited insight into feature importance.

Selection Implication: Prioritize TCN/SCINet only if capturing long-term/multi-scale dependencies is critical and resources allow. Simpler CNNs or other models (e.g., RNNs for moderate lengths) may be preferable for efficiency or interpretability.

C. METHODS BASED ON RECURRENT NEURAL NETWORKS

1) RNN

Recurrent Neural Networks (RNNs) are a type of deep learning model proposed by Jordan in 1990, specifically designed to handle data with temporal characteristics [21]. The design philosophy of RNNs is based on the idea that “human cognition relies on past experiences and memories,” enabling them to process temporal dependencies in time series data by memorizing historical information. Initially, RNNs saw widespread application in natural language processing tasks, like language modeling and machine translation. However, they have also demonstrated outstanding performance in time series prediction tasks, especially in scenarios that demand the capture of long-term dependencies. The most basic structure of an RNN is shown in Figure 4, consisting of four components: the input x_i , the output y_i , the hidden state h_i , and the RNN cell [22].

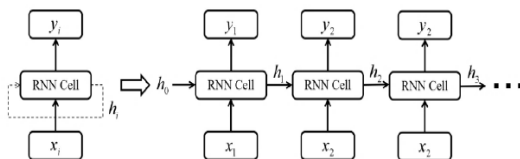


FIGURE 4. Structure of a simple RNN.

RNNs are naturally suited for sequential data by maintaining a hidden state that acts as a memory of past inputs. Standard RNNs process sequences step-by-step, allowing them to model temporal order explicitly. However, they are prone to the vanishing/exploding gradient problem, which severely limits their ability to capture long-term dependencies.

2) LSTM AND GRU: GATED ARCHITECTURES

While RNNs excel at processing time series data, they encounter difficulties in capturing long-term dependencies

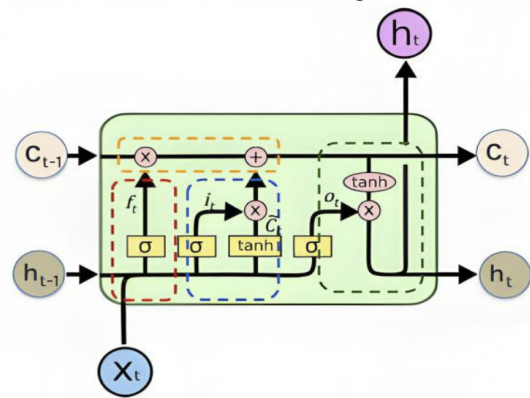


FIGURE 5. Structure of LSTM.

due to issues like vanishing gradients and exploding gradients. In order to tackle this problem, Hochreiter et al. introduced the Long Short-Term Memory (LSTM) network back in 1997 [23]. LSTM is an extension of RNN that effectively captures and stores long-term dependencies by introducing a “gating mechanism.” LSTM can extract important features from input data and retain this information over extended periods, making it particularly effective in long-term time series prediction tasks. The fundamental structure of LSTM is composed of three gating units: the Forget Gate, the Input Gate, and the Output Gate. These gates are responsible for controlling the forgetting, updating, and outputting of information, respectively. The structure of LSTM is illustrated in Figure 5.

The Gated Recurrent Unit (GRU) is an improved model proposed based on the Long Short-Term Memory (LSTM) network. GRU retains the core idea of LSTM, i.e., capturing long-term dependencies through gating mechanisms, but its network structure is more simplified. GRU integrates the “input gate” and “forget gate” of LSTM into a single “update gate” and adds a “reset gate” to manage the flow of information. This design not only reduces the number of model parameters and computational complexity but also mitigates the vanishing and exploding gradient problems to some extent. [24]. The structure of GRU [25] is shown in Figure 6.

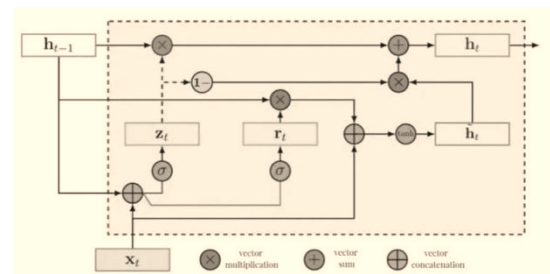


FIGURE 6. Structure of GRU.

Long Short-Term Memory (LSTM) networks overcome the gradient limitations of RNNs through a sophisticated gating mechanism (Forget Gate, Input Gate, Output Gate)

and a memory cell, enabling selective retention and updating of information over long sequences. Gated Recurrent Units (GRU) offer a streamlined alternative by merging the forget and input gates into an update gate and introducing a reset gate, reducing computational complexity while often achieving comparable performance to LSTM.

LSTM/GRU have been widely applied to various time series forecasting tasks, including network traffic prediction [26], power load forecasting [27], equipment failure prediction [28], electric vehicle charging demand prediction [29], traffic flow prediction [30], and e-commerce sales forecasting during promotions [31], [32].

3) COMPARATIVE ANALYSIS OF RNN-BASED ALGORITHMS

Recurrent neural network-based algorithms have long been essential methods for solving time series prediction tasks. They are often embedded as modules in other algorithms to achieve better prediction performance and served as the primary models for time series data prediction before 2017, with widespread applications. A comparison analysis of RNN-based algorithms is summarized in Table 2.

TABLE 2. Analysis of RNN-based algorithms.

Algorithm	Strengths	Limitations
RNN	-Strong temporal dependency modeling capability	-Vanishing/exploding gradient issues
	-High flexibility	-Low computational efficiency
LSTM	-Simple structure	-Limited ability to handle long sequences
	-Strong long-term dependency modeling capability	-High computational complexity
	-Mitigates gradient problems	-Large number of parameters
	-High flexibility	-Training difficulty
GRU	-Excellent performance	-Limited expressive power
	-Simplified structure	-Sensitivity to data
	-High training efficiency	-Limited ability to handle long sequences
	-Strong long-term dependency modeling capability	
	-Suitable for small/medium-scale data	

RNN: A foundational model suitable for short-term time series prediction tasks, especially in scenarios with prominent local features (e.g., short-term stock price fluctuations, sensor data prediction). However, it struggles with long sequences and gradient issues.

LSTM: Addresses RNN’s gradient problems through gating mechanisms, making it suitable for long-sequence prediction tasks (e.g., power load forecasting, weather data prediction) where long-term dependencies need to be captured. Its drawbacks include high computational complexity.

GRU: A simplified version of LSTM, ideal for small/medium-scale datasets or resource-constrained scenarios (e.g., traffic flow prediction, promotional sales forecasting). While efficient, its performance may lag behind LSTM in complex tasks.

The evolution from RNNs to LSTM/GRU involves fundamental trade-offs:

Memory vs. Cost/Parallelism: Gating mechanisms (LSTM’s 3 gates, GRU’s 2 gates) effectively solve the vanishing gradient problem, enabling long-term memory. This comes at a high computational cost per step and inherently sequential processing, severely limiting training parallelism and speed compared to CNNs/Transformers. GRU offers a lower-cost alternative to LSTM with comparable performance in many cases.

Flexibility vs. Overfitting/Data Hunger: Their general-purpose sequential modeling capability is versatile. However, this power increases overfitting risk (especially for smaller datasets) and often demands more data than simpler models to achieve robustness.

Selection Implication: Choose LSTM/GRU when modeling order and causality is paramount for medium/long dependencies, and sequential processing is acceptable. GRU is often the pragmatic default; use LSTM if maximum representational power is needed. Consider alternatives (TCN, Transformer) for very long sequences or when parallel speed is critical.

D. METHODS BASED ON TRANSFORMER

1) TRANSFORMER

The Transformer model was introduced by Vaswani et al. in 2017 and was initially utilized in natural language processing tasks. Its core idea relies entirely on the Self-Attention Mechanism to capture long-range dependencies in sequences [33]. Unlike traditional Recurrent Neural Networks (RNNs) and Convolutional Neural Networks (CNNs), the Transformer discards recurrent and convolutional operations, depending solely on attention mechanisms for sequence modeling. This design not only significantly enhances parallel computing capabilities but also enables processing of longer sequences while dramatically improving training and inference efficiency.

The overall architecture of the Transformer follows an Encoder-Decoder framework [34], as illustrated in Figure 7. Both the encoder and decoder consist of multiple stacked layers, each containing Multi-Head Self-Attention mechanisms and Feed-Forward Neural Networks.

The Transformer architecture revolutionized sequence modeling by relying solely on the self-attention mechanism, dispensing with recurrence and convolution. This allows it to capture long-range dependencies effectively and enables massive parallelization during training. However, the self-attention mechanism has quadratic time and space complexity ($O(L^2)$) with respect to sequence length L , posing a significant challenge for very long time series forecasting.

2) INFORMER

While the Transformer model exhibits powerful sequence modeling capabilities, its self-attention mechanism suffers from quadratic growth in time and space complexity as

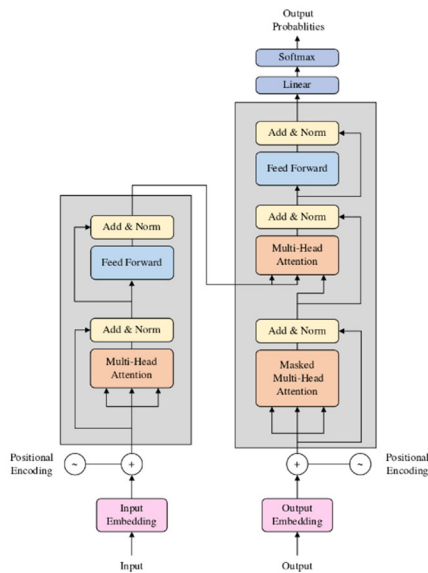


FIGURE 7. Structure of transformer.

the sequence length increases, limiting its effectiveness in long-sequence prediction tasks [35]. To address this, researchers proposed the Informer model for long-sequence forecasting. Informer introduces three key innovations:

ProbSparse Self-Attention Mechanism: Reduces the time and space complexity from $O(L^2)$ to $O(L \log L)$, where L is the sequence length.

Self-Attention Distilling Mechanism: Shortens the input sequence length by distilling dominant features, thereby reducing computational and memory costs.

Generative-Style Decoder: Directly generates predictions in a single forward step during inference, rather than using a step-by-step autoregressive approach, significantly reducing prediction time.

The overall architecture of the Informer model [36] is illustrated in Figure 8.

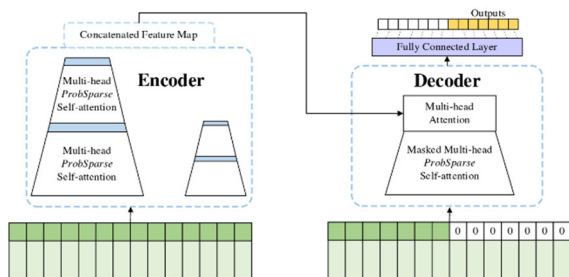


FIGURE 8. Structure of informer.

The overall architecture of Informer retains the encoder-decoder structure. The encoder is mainly made up of stacked ProbSparse Self-Attention Layers and Self-Attention Distilling Layers. The decoder is largely composed of ProbSparse Self-Attention and Multi-Head Self-Attention mechanisms. Lastly, the output from the decoder is further processed

via a fully connected layer to generate the final prediction outcomes.

3) AUTOFORMER

Prior to the proposal of Autoformer, Transformer-based models primarily relied on self-attention mechanisms to capture long-term dependencies in sequences. However, when handling complex time series data with long-term future horizons, traditional Transformer models often struggle to effectively capture reliable dependencies. Additionally, although Informer improves the efficiency of long-sequence processing through sparse self-attention mechanisms, its information utilization still faces bottlenecks. To address these issues, researchers proposed Autoformer, a novel architecture based on an Auto-Correlation Mechanism and series decomposition. Autoformer embeds series decomposition into the model's fundamental modules, endowing it with the capability for progressive decomposition of complex time series. Simultaneously, it replaces the traditional self-attention mechanism with the auto-correlation mechanism, significantly enhancing information utilization efficiency [37]. The overall structure of Autoformer is illustrated in Figure 9.

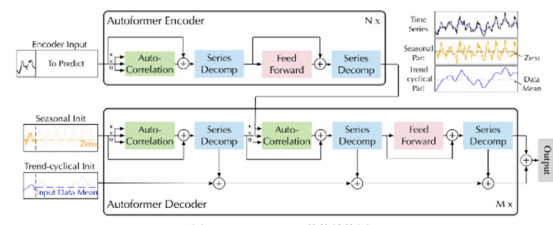


FIGURE 9. Structure of autoformer.

The encoder and decoder parts of Autoformer both include three modules: the auto-correlation mechanism, series decomposition, and feed-forward networks. The encoder focuses on modeling the seasonal components of the sequence, and the output containing past seasonal components is provided as cross-information to the decoder for prediction processing. The decoder is mainly divided into two parts: cumulative operations for the trend-cyclical components and stacked auto-correlation mechanisms for the seasonal components output by the encoder [38]. Each decoder layer contains an inner auto-correlation mechanism and an encoder-decoder auto-correlation mechanism, which respectively help the model refine predictions and extract seasonal characteristics from past sequences.

4) ITRANSFORMER

iTransformer was proposed by Liu et al. [39], iTransformer fundamentally rethinks how multivariate time series are processed within the Transformer framework. Unlike traditional approaches that treat time points as tokens and variables as features (channel dimensions), iTransformer inverts the dimensions: it treats each variable as a token and embeds the temporal points of that variable as the token's

features. The overall structure of iTransformer is illustrated in Figure 10. Key innovations include:

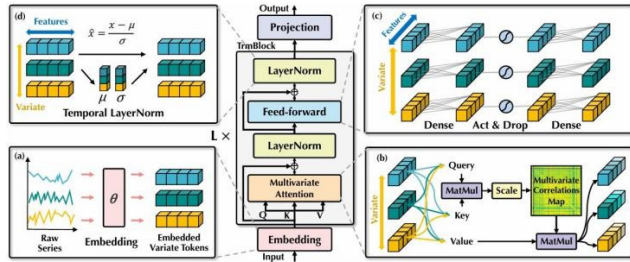


FIGURE 10. Structure of iTransformer.

(a) Variable-as-Token: Each multivariate series is transformed into a sequence of tokens, where each token represents one variable. The token embedding contains the entire historical sequence of that single variable.

(b) Multivariate Self-Attention: The self-attention mechanism operates directly on these variable tokens, learning the complex dependencies and interactions between different variables. This is a stark contrast to models using channel-independent embeddings or simple concatenation.

(c) Temporal Modeling via Feed-Forward Networks (FFNs): The powerful FFN layers within each Transformer block are then responsible for modeling the intricate temporal patterns within each individual variable's sequence. The inverted structure allows the FFNs to focus solely on the temporal dimension per variable.

iTransformer leverages the Transformer's token relationships and FFN's non-linear fitting to effectively capture cross-variable correlations while maintaining temporal modeling. It significantly outperforms models like Autoformer/FEDformer [40] in multivariate forecasting due to its simplicity. Gains are less pronounced on univariate tasks, and it represents a paradigm shift prioritizing variable dimensions.

5) OTHER TRANSFORMER-BASED METHODS

With the success of Transformer models in natural language processing, an increasing number of studies have applied them to time series prediction tasks [41]. Below are several improved Transformer-based models and their applications in time series prediction.

In 2020, Wu et al. [42] proposed the Adversarial Sparse Transformer (AST). Based on Sparse Transformer [43], AST incorporates ideas from Generative Adversarial Networks (GANs) to enhance prediction performance through adversarial training. AST adopts an encoder-decoder architecture, enabling high-fidelity multi-step predictions at the sequence level. Experiments show that dependencies between time series steps exhibit sparsity, and AST improves prediction performance globally through adversarial training. Compared to traditional autoregressive decoders, the encoder-decoder-based AST demonstrates superior accuracy and stability.

In 2023, Li et al. [44] proposed Conformer, a Transformer model tailored for multivariate long-period time series

prediction. Conformer extracts correlation features from multivariate time series using Fast Fourier Transform (FFT), effectively modeling relationships between multiple variables and capturing patterns at different time scales (e.g., monthly, weekly, daily, hourly). To enhance long-period prediction efficiency, Conformer employs a sliding window method, where each position computes attention weights only with neighboring nodes within the window, reducing time complexity to $O(L)$. Furthermore, Conformer introduces stationary and instant recurrent network modules, encoding input time series with GRU to compensate for global information loss caused by the sliding window method, further boosting prediction capabilities.

In 2023, Kim et al. proposed Scaleformer [45], it addresses multi-scale temporal dependencies through a dual adaptive mechanism comprising: (1) scale-aware attention that dynamically selects optimal attention granularity (e.g., hourly/daily/weekly bins), and (2) gated resource allocation that learns to distribute computational resources between local and global attention heads. This architecture represents a significant advancement in computational efficiency for ultra-long-horizon forecasting, though its predefined scale bins may constrain adaptability to irregular temporal patterns.

In 2023, Zhang et al. developed CrossFormer [46]. CrossFormer introduces a novel cross-scale attention mechanism that explicitly models multi-periodicity through: (1) hierarchical feature extraction at predefined temporal scales (e.g., 24h/168h/720h), (2) learnable fusion of periodic components via gating mechanisms, and (3) phase-aware alignment for drifting seasonality. This architecture provides state-of-the-art modeling of complex periodic patterns, though its fixed scale definitions may limit effectiveness for irregularly sampled or non-stationary time series.

6) COMPARATIVE ANALYSIS OF TRANSFORMER-BASED ALGORITHMS

Transformer-based algorithms are now widely used in various artificial intelligence tasks. Building models on the Transformer framework can break through the limitations of traditional algorithms, enabling simultaneous capture of short-term and long-term dependencies, effectively solving long-sequence prediction challenges, and supporting parallel processing. A comparison analysis of transformer-based algorithms is summarized in Table 3.

Transformer-based algorithms have demonstrated remarkable capabilities in long-range dependency modeling, though they present distinct trade-offs. The Informer architecture achieves efficient long-sequence processing through its ProbSparse attention mechanism, while Autoformer enhances periodicity modeling via series decomposition and autocorrelation mechanisms. Newer variants address specific limitations: iTransformer's inverted architecture (channel-independent processing) significantly improves multivariate forecasting robustness but sacrifices short-term feature sensitivity; Scaleformer's adaptive computation allocation

TABLE 3. Analysis of transformer -based algorithms.

Algorithm	Strengths	Limitations
Transformer	- Strong long-range dependency modeling capability - High parallel computing capability - High flexibility	- High computational complexity - Large data requirements - Lacks positional information
Informer	- Efficient long-sequence processing capability - Fast prediction speed - Broad applicability - Progressive series decomposition	- High model complexity - Sensitive to data quality
Autoformer	- Auto-correlation mechanism - Broad applicability - Inverted dimensions enable efficient multivariate dependency modeling	- Less pronounced gains on univariate tasks - Redesign requires rethinking temporal-variable prioritization
iTransformer	- FFNs focus on temporal patterns per variable - State-of-the-art performance on multivariate benchmarks - Adversarial training	- High computational complexity
AST	- High-fidelity multi-step prediction - Global optimization capability - Fast Fourier Transform (FFT) feature extraction - Sliding window method for reduced complexity	- Training difficulty - Potential global information loss - High model complexity
Conformer	- Multi-scale temporal modeling capability - Adaptive multi-scale processing	- Weaker long-term trend capture
Scaleformer	- Optimized computation allocation - State-of-the-art long-horizon efficiency - Explicit multi-periodicity modeling	- Training instability with small datasets
CrossFormer	- Phase-aware seasonality alignment - Strong performance on cyclical data	- Fixed scale definitions limit flexibility - High memory footprint for fine resolutions

optimizes ultra-long-horizon efficiency at the cost of predefined scale constraints; CrossFormer provides state-of-the-art multi-periodicity modeling through explicit cross-scale attention, though its fixed temporal bins limit irregular pattern adaptability.

For applications requiring extreme sequence lengths (>10,000 steps), Scaleformer currently provides the optimal efficiency-accuracy balance. When handling high-dimensional multivariate series with independent channels (e.g., multi-sensor industrial systems), iTransformer demonstrates superior noise robustness. CrossFormer excels in domains with strong inherent periodicities (epidemiology, energy demand), while the original Transformer architecture

maintains advantages in general sequence-to-sequence tasks with moderate lengths.

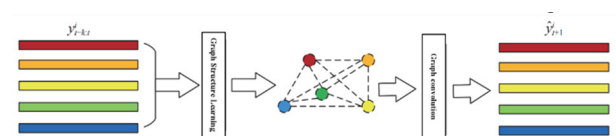
Interpretability remains challenging across all Transformer variants. While attention visualization offers partial insight, models increasingly rely on post-hoc methods like SHAP [47] and LIME [48] for feature attribution.

SHAP: A game-theoretic approach that assigns each feature (variable at a specific time) an importance value for a particular prediction. It is highly valuable for quantifying the contribution of different drivers in multivariate forecasts (e.g., “What was the impact of temperature vs. humidity on the predicted energy load?”).

LIME: Approximates the complex model locally around a specific prediction with a simpler, interpretable model (e.g., linear regression). This helps in understanding the model’s behavior for individual forecasts (e.g., “Why did the model predict a spike in traffic at this particular hour?”).

E. METHODS BASED ON GRAPH NEURAL NETWORKS

In multivariate time series forecasting, variables often exhibit complex interdependencies that traditional methods may struggle to capture effectively. In recent years, Graph Neural Networks (GNNs) [49] have demonstrated powerful capabilities in modeling relational dependencies. GNNs address this by representing time series data as graph structures, where graph convolution operations capture spatial dependencies between variables, and temporal convolution operations model temporal dependencies within the sequences. This dual mechanism enables effective prediction of multivariate time series [50]. Recent advances further extend this paradigm through spatiotemporal frameworks (e.g., MTGNN, STGCN) that unify spatial and temporal modeling in an end-to-end architecture. The basic framework of GNNs is illustrated in Figure 11.

**FIGURE 11. Structure of GNN.**

1) MAIN CATEGORIES OF GRAPH NEURAL NETWORK-BASED METHODS

(a) Spatial Graph Convolution-Based Methods

This approach utilizes graph convolution operations to capture spatial dependencies in time series data. By transforming time series data into graph structures, graph convolution propagates information across the graph, integrating features from neighboring nodes. The advantage of spatial graph convolution is its ability to effectively capture local spatial dependencies, making it well-suited to data with explicit spatial structures. However, it also has limitations, such as lower computational efficiency for large-scale graph data and challenges in handling dynamic graph structures.

(b) Temporal Convolution-Based Methods

Temporal convolution methods primarily focus on extracting temporal dependencies through temporal convolution modules. These methods excel at processing sequential information in time series data and perform well in predicting data with significant temporal dynamics. However, they may overlook spatial dependencies in the data, leading to suboptimal performance when dealing with complex spatial structures.

(c) Spatial-Temporal Integrated Methods

This approach constructs a spatiotemporal parallel feature extraction framework that simultaneously considers both spatial and temporal dependencies in time series data. For example, emerging frameworks like STGCN (Spatio-Temporal Graph Convolutional Network) and MTGNN (Multivariate Time Series Graph Neural Network) jointly model spatial and temporal dependencies through:

STGCN (Spatio-Temporal Graph Convolutional Network) is a graph neural network framework specifically designed for spatiotemporal data with fixed topology. Its core innovation lies in the cascaded architecture of spatial-temporal convolutions. The method adopts a sandwich structure (GCN-TCN-GCN), where graph convolutional networks (GCN) in the spatial dimension capture spatial dependencies between nodes (e.g., connections between sensors in traffic networks), while dilated causal convolutions (TCN) in the temporal dimension extract multi-scale temporal features [51].

MTGNN (Multivariate Time Series Graph Neural Network) is a graph neural network framework that is specifically tailored for multivariate time series forecasting. Its core innovation lies in end-to-end dynamic graph structure learning and joint spatiotemporal dependency modeling. The method automatically infers implicit relationships between variables through trainable node embeddings, generating dynamic adjacency matrices that overcome the limitations of traditional GNNs relying on prior knowledge. For temporal modeling, MTGNN employs dilated causal convolution to capture multi-scale temporal patterns, combined with gating mechanisms to filter noise [52].

Compared to approaches that model spatial and temporal dependencies separately, these integrated methods often demonstrate superior performance on benchmarks.

2) TIME SERIES PREDICTION RESEARCH BASED ON GRAPH NEURAL NETWORKS

In recent years, the application of Graph Neural Networks (GNNs) in time series prediction has grown significantly, particularly in multivariate time series forecasting [53]. By modeling spatial dependencies between variables, GNNs have substantially improved prediction performance. An increasing number of scholars are now applying GNNs to time series prediction research.

Ariele and Brunod et al. [54] proposed a Graph Convolutional Recurrent Neural Network (GCRNN) to predict water demand in supply systems within the same geographical region. This method constructs a graph structure

to link water demand time series across different regions, capturing both spatial and temporal dependencies. Specifically, GCRNN integrates spatial information through graph convolution operations and captures temporal dynamics via recurrent neural networks. Experiments demonstrate that GCRNN effectively predicts water demand, providing reliable support for water resource management.

Yu et al. proposed a deep learning framework based on Spatio-Temporal Graph Convolutional Network (STGCN) for traffic flow forecasting. Addressing the issue that traditional methods struggle to capture complex spatio-temporal dependencies in mid-to-long-term predictions, STGCN constructs a fully convolutional structure of spatio-temporal convolution blocks by combining graph convolution with gated temporal convolution, effectively integrating the topological information of the traffic network and dynamic temporal features. The research results verify the efficiency and scalability of spatio-temporal graph convolution in traffic prediction, providing a practical solution for intelligent transportation systems.

Wu et al. proposed an end-to-end framework named MTGNN for modeling dynamic spatio-temporal dependencies. The innovation of this method lies in its ability to automatically learn the dynamic graph structure among variables through trainable node embeddings, without the need for a predefined adjacency matrix. It also captures multi-scale temporal patterns by combining dilated causal convolution with gating mechanisms. Experiments show that its training efficiency and generalization ability are superior to traditional graph neural networks, providing a general solution for multivariate time series prediction without explicit structure.

Lan et al. proposed a dynamic spatial-temporal (DSTAGNN) aware graph based on a data-driven strategy to replace the predefined static graph commonly used in traditional graph convolution. They considered that historical data contains the intrinsic dynamic information of the road network's spatial structure. Additionally, the paper designed a new graph neural network architecture. This architecture can not only represent the dynamic spatial correlation between nodes with an improved multi-head attention mechanism but also capture extensive dynamic temporal correlation from multi-receptive-field features through multi-scale gated convolution. Extensive experiments on real-world datasets show that the performance of the proposed algorithm has been improved [55].

HGCN-MA (Hierarchical Graph Convolution with Multi-scale Attention) proposed by Wang et al. [56], this architecture captures multi-granularity spatio-temporal dependencies through three key innovations: (1) hierarchical graph pooling and up-sampling operations that aggregate regional patterns at different spatial resolutions, (2) multi-scale attention mechanisms that dynamically fuse hourly, daily, and weekly temporal features, and (3) adaptive edge weighting that models evolving relationships between nodes. This hierarchical approach effectively captures regional-to-local dependencies in urban systems, though its fixed pooling ratios

TABLE 4. Analysis of GNN -based algorithms.

Algorithm	Strengths	Limitations
Spatial Graph Convolution-Based Methods (e.g., GCN, GAT)	- Explicitly models spatial dependencies between variables (e.g., sensor networks, traffic nodes). - Effective for data with fixed or known graph structures. - High interpretability due to predefined adjacency matrices.	- Requires prior knowledge of graph topology (e.g., road networks). - Struggles with dynamic or unknown relationships. - Computationally expensive for large graphs ($O(N^2)$ complexity).
Temporal Convolution-Based Methods (e.g., TCN, T-GCN)	- Efficiently captures long-term temporal dependencies using dilated convolutions. - Parallelizable architecture reduces training time. - Robust to irregular sampling intervals.	- Ignores spatial correlations between variables. - May fail to model complex spatiotemporal interactions. - Limited adaptability to non-stationary patterns.
Spatial-Temporal Integrated Methods (e.g., STGCN, MTGNN)	- Jointly models spatial and temporal dependencies end-to-end. - STGCN: Lightweight and efficient for fixed-topology data (e.g., traffic forecasting). - MTGNN: Automatically infers dynamic graph structures, ideal for evolving systems (e.g., stock markets).	- STGCN: Limited to static graph topologies. - MTGNN: Higher computational cost due to graph learning. - Both require careful hyperparameter tuning for optimal performance.

require careful hyperparameter tuning for optimal spatial partitioning.

STAN (Spatio-Temporal Attention Network) developed by Chen et al. [57], STAN introduces a dual-path isolation framework featuring: (1) a spatial path using graph isomorphism networks with edge gating to filter irrelevant connections, (2) a temporal path employing dilated causal convolutions with adaptive receptive fields that automatically adjust to changing event durations, and (3) a cross-attention fusion module that learns interaction weights between spatio-temporal features. The dual-path isolation provides exceptional modeling fidelity for critical phenomena like natural disasters, though its multi-stage training protocol demands significant computational resources.

3) COMPARATIVE ANALYSIS OF GNN-BASED ALGORITHMS

Graph Neural Networks (GNNs) have emerged as powerful tools for modeling relational dependencies in time series data. By combining spatial and temporal modeling capabilities, GNN-based methods offer unique advantages for various prediction tasks. A comparative analysis of three key GNN approaches is summarized in Table 4.

Spatial methods excel in structured environments, while temporal methods prioritize sequence modeling. Integrated

approaches (e.g., MTGNN) bridge this gap but increase complexity.

Spatial GCNs: Ideal for traffic/weather prediction with fixed sensors.

Temporal CNNs: Best for univariate or spatially independent series.

Integrated Methods: Essential for dynamic systems (e.g., finance, IoT).

Recent spatio-temporal GNN innovations focus on dynamic relationship modeling and multi-scale aggregation. HGCN-MA's hierarchical approach effectively captures regional-to-local dependencies. For high-stakes applications like disaster forecasting, STAN's dual-path isolation of spatial and temporal features provides critical modeling fidelity.

F. HYBRID MODELS: INTEGRATING DEEP LEARNING WITH STATISTICAL METHODS

Recent studies have demonstrated that combining deep learning models with traditional statistical methods or domain-specific constraints can significantly enhance prediction accuracy and robustness [58]. Hybrid approaches leverage the strengths of both paradigms—statistical models for capturing linear patterns and deep learning for modeling nonlinear dynamics—providing a more comprehensive solution to complex time series forecasting tasks.

1) STATISTICAL-DEEP LEARNING INTEGRATION

This category integrates classical statistical forecasting models (e.g., ARIMA, ETS) with deep neural networks to leverage their complementary strengths.

ARIMA-RNN Hybrid [59] combines ARIMA for linear component modeling with RNNs for nonlinear residual processing. The ARIMA module first captures linear trends and seasonality, while the residuals (nonlinear components) are fed into an RNN for refinement. On the Electricity Load Dataset, this hybrid achieved 15% MAE reduction compared to standalone models. This approach exemplifies effective error-correction synergy between statistical and deep learning paradigms.

ETSformer [60] integrates exponential smoothing state space models (ETS) into Transformers through two innovations: i) Exponential Smoothing Attention replacing standard self-attention for efficient recurrence, and ii) built-in seasonal-trend decomposition layers. It reduces inference latency by 37% vs. Autoformer on ETT data while maintaining accuracy. This framework sets new standards for interpretable and efficient statistical-neural fusion.

2) DECOMPOSITION-BASED HYBRID MODELS

These models explicitly decompose time series into interpretable components (trend, seasonality, residuals) using statistical or learnable methods, with deep networks modeling residuals or sub-components.

BEATS [61] employs iterative decomposition blocks where each block: i) extracts features via FCN, ii) projects

interpretable trend (polynomial) and seasonal (harmonic) components, and 3) passes residuals downstream. The interpretable variant (N-BEATS-I) won the M4 competition. Its hierarchical decomposition provides unprecedented accuracy and explainability for heterogeneous time series.

PatchTST [62] introduces patching - splitting series into overlapping sub-series as Transformer tokens - enabling efficient long-horizon modeling. When combined with statistical methods (e.g., ARIMA for patch-level trends), it achieves 23% lower MSE than Informer with 60% less GPU memory. Its segment-wise processing redefines efficiency for ultra-long sequences while maintaining compatibility with statistical techniques.

3) DOMAIN-CONSTRAINED HYBRID MODELS

Incorporates physical laws, operational constraints, or domain knowledge as inductive biases or loss functions within deep architectures.

Physics-Informed Hybrids [63] integrate domain knowledge via dynamically weighted ensembles of physics-based simulators (“experts”) and deep learners. For power load forecasting, they explicitly model interactions between seasonal trends and external factors like temperature/economic indicators. This approach guarantees physical plausibility while capturing complex nonlinear dynamics.

GNN-Statistical Hybrids combines Graph Neural Networks with statistical models (e.g., VAR, structural equations), these hybrids enforce explicit relational constraints in multivariate forecasting. Examples include supply-chain networks where GNNs capture spatial dependencies while VAR models temporal causality. They provide auditable relational reasoning critical for high-stakes domains like economics and epidemiology.

3) COMPARATIVE ANALYSIS OF HYBRID MODELS

Hybrid models, which synergistically integrate deep learning with statistical methods or domain knowledge, have emerged as a powerful paradigm for time series prediction. A comparative analysis of the hybrid models is summarized in Table 5.

Hybrid models significantly enhance prediction accuracy and robustness by synergistically integrating statistical methods (e.g., linear patterns, interpretable decomposition) with deep learning (nonlinear dynamics, automatic feature learning). However, they involve critical trade-offs across three dimensions:

a: ACCURACY-RICHNESS VS. COMPLEXITY

Decomposition-based hybrids (e.g., N-BEATS, Autoformer, PatchTST) excel in capturing multi-scale components but incur computational redundancy from iterative decomposition (N-BEATS) or patching operations (PatchTST).

Stat-DL integrations (e.g., ETSformer, ARIMA-RNN) improve interpretability through explicit statistical foundations (e.g., exponential smoothing in ETSformer), yet depend heavily on underlying assumptions (stationarity, known seasonality).

TABLE 5. Analysis of hybrid models.

Algorithm	Strengths	Limitations
ARIMA-Deep Learning Integration (eg. ARIMA-RNN, ETSformer)	- Explicit linear-nonlinear synergy - Error correction mechanism - ETSformer: Enhanced efficiency & interpretability	- High complexity - Dependence on statistical assumptions (ARIMA/ETS) - Limited global optimization - Error propagation risk
Decomposition-Based Hybrid Models (eg. N-BEATS, PatchTST)	- Multi-scale component separation (Autoformer, N-BEATS) - Enhanced interpretability (N-BEATS-I) - PatchTST: Efficient long-horizon modeling - N-BEATS: SOTA benchmark performance	- Computational redundancy (iterative) - Potential ignored component interactions - PatchTST: Patching hyperparameter sensitivity
Domain-Constrained Hybrid Models (eg. Physics-Informed, GNN-VAR)	- Physical consistency/plausibility - Improved data efficiency - Enhanced robustness	- High domain expertise requirement - Reduced flexibility - Increased implementation complexity

b: DOMAIN COMPLIANCE VS. FLEXIBILITY

Domain-constrained models enforce physical plausibility (e.g., energy conservation in Physics-Informed Hybrids) or relational causality (GNN-VAR), but require significant expert knowledge for constraint design and risk over-regularization.

c: EFFICIENCY CHALLENGES

While PatchTST reduces GPU memory by 60% via sub-series tokenization and ETSformer cuts inference latency by 37% via recurrence, most hybrids (e.g., N-BEATS with 30+ blocks) remain computationally intensive versus end-to-end deep models.

Future Directions:

Automated Fusion: End-to-end joint training (e.g., differentiable ARIMA layers) to replace staged pipelines.

Lightweight Decomposition: Integrated architectures like PatchTST’s tokenization that avoid iterative residuals.

Universal Constraints: Template libraries for domain knowledge (e.g., physics loss functions) to reduce customization overhead.

IV. SYSTEMATIC EVALUATION AND TASK-DRIVEN MODEL SELECTION

A. COMPARATIVE ANALYSIS OF MODEL CATEGORIES

Time series forecasting plays a critical role in numerous fields, with different types of neural network models exhibiting distinct characteristics. This section analyzes the strengths and limitations of the five major model categories discussed (CNN-based, RNN-based, Transformer-based, GNN-based, Hybrid Models) and provides a comprehensive

TABLE 6. Comparative analysis of model categories.

Model Category	Strengths	Limitations
CNN-based	- Strong local feature extraction capability	- Limited long-term dependency modeling
	- Parameter sharing reduces computational cost	- Insensitivity to data sequence order
RNN-based	- Translation invariance	- Training difficulties
	- Specialized for sequential data processing	- High computational complexity
Transformer-based	- Strong long-term dependency modeling capability	- Prone to overfitting
	- High flexibility	- High computational resource requirements
GNN-based	- Powerful global dependency modeling capability	- Lack of explicit temporal order modeling
	- Strong parallel computing capability	- Limited interpretability
Hybrid Models	- High flexibility and scalability	- High computational complexity
	- Strong capability for modeling complex relational structures	- Heavy dependency on data quality and structure
	- Adaptability to non-Euclidean data	- Training challenges
	- Relatively better interpretability	- Complexity and Computational Cost
	- Complementary Modeling Capabilities	- Assumption Dependency
	- Domain Knowledge Integration	- Flexibility and Generalizability
	- Enhanced Interpretability	Constraints
	- Data Efficiency and Generalization	- Decomposition Errors and Redundancy

comparison of their key attributes, summarized in Tables 6 and 7.

Generally speaking, CNN-based models are suitable for short-term time series prediction tasks (e.g., short-term stock price fluctuations, sensor data prediction), particularly in scenarios with prominent local features. RNN-based models excel in long-sequence prediction tasks (e.g., power load forecasting, speech recognition), effectively capturing long-term dependencies. Transformer-based models are ideal for long-sequence and global dependency modeling tasks (e.g., machine translation, text generation), leveraging their powerful parallel computing capabilities. GNN-based models are well-suited for multivariate time series prediction tasks (e.g., traffic flow forecasting, social network analysis), especially in scenarios where variables exhibit complex interdependencies. Hybrid models are suitable for complex temporal tasks that require simultaneous capture of local features and long-term dependencies (e.g., combining CNNs with ARIMA to handle short-term fluctuations under high-frequency noise) or integration of domain knowledge with data-driven modeling (e.g., GNNs incorporating VAR for multivariate spatial correlation analysis).

Based on table 8, to address the gap in practical model selection, we propose a task-driven framework (Figure 12) that maps key problem characteristics to optimal model classes:

- For short-to-medium sequences (<1000 steps) with prominent local patterns:
Prioritize CNN/TCN models (e.g., ECG anomaly detection).
If temporal order is critical, use lightweight RNN variants (GRU).
- For long sequences (>1000 steps):
Adopt Transformer architectures (e.g., Informer for efficiency, Autoformer for seasonal data).
If computational resources are limited, consider TCN as an alternative.
- For multivariate series with interdependencies:
Select GNN-based models (e.g., MTGNN for dynamic graphs, STGCN for fixed topologies).
For variable-specific temporal patterns, iTransformer is preferred.
- Under strict latency/memory constraints:
Choose GRU or Kmeans-CNN for small-scale data.
Employ hybrid models (e.g., PatchTST+ARIMA) for resource-efficient long-horizon forecasting.
Example: In traffic flow prediction (long sequences + spatial dependencies), STGCN or MTGNN outperforms pure temporal models. Conversely, for high-frequency stock volatility (short sequences + low latency needs), TCN is optimal. This framework bridges theoretical capabilities with pragmatic deployment considerations.
Selecting a forecasting model also requires navigating core, often competing, priorities:
Modeling Power vs. Efficiency: High capacity for complex/long-range patterns (Transformers, GNNs, deep TCN/LSTM) demands significant computation. Simpler models (basic CNN, linear) are efficient but limited.
Sequential Fidelity vs. Parallel Speed: RNNs (LSTM/GRU) naturally handle order/causality but train slowly (sequential). CNNs/Transformers leverage parallelism for speed but require explicit mechanisms (causal convolutions, masking) for causality.
Automatic Learning vs. Interpretability: Deep models excel at feature learning but are “black boxes.” Simpler statistical models or hybrids offer more transparency but need manual feature engineering or lack flexibility.
Generalization vs. Specialization: General models (LSTM/GRU, vanilla Transformer) need more data. Specialized models (Autoformer for seasonality, SCINet for trends, GNNs for relational data) can be data-efficient if their bias matches the data, but fail if assumptions break.
There is no universally optimal model. Selection should be guided by matching the model’s strengths to the specific problem requirements and constraints. Prioritizing the most critical trade-offs for a given task is essential. Table 8 and the preceding category-specific analyses serve as valuable frameworks to inform this decision-making process.

TABLE 7. Multi-dimensional comparison of deep learning models for time series forecasting.

Model Category	Sequence Length Support	Multivariate Support	Interpretability	Computational Cost	Real-time Performance	Industrial Readiness	Typical Application Scenarios
CNN-based	Short-Medium (★★☆)	Medium (★★☆)	Low (★☆☆)	Low (★★★★)	High (★★★★)	High (★★★★)	High-frequency financial data, Sensor monitoring
RNN-based	Medium-Long (★★☆)	High (★★★★)	Medium (★★☆)	Medium (★★☆)	Medium (★★☆)	High (★★★★)	Power load forecasting, Speech recognition
Transformer	Very Long (★★★★)	High (★★★★)	Low (★☆☆)	High (★☆☆)	Low (★☆☆)	Medium (★★☆)	Climate prediction, Cross-year sales trend analysis
GNN-based	Medium (★★☆)	Very High (★★★★)	Medium (★★☆)	High (★☆☆)	Medium (★★☆)	Low (★☆☆)	traffic network prediction, Social contagion modeling
Hybrid Models	Varies	Varies	High (★★★★)	Varies (Often High)	Varies (Often Low)	Medium (★★☆)	Domain-specific tasks with known structure

Rating Explanation:
★★★★: Strong support / High efficiency / Low cost
★★☆: Moderate support
★☆☆: Weak support/High cost
Sequence Length: Short (<100 steps), Medium (100-1000 steps), Long (>1000 steps)
Computational Cost: Based on GPU memory consumption and inference latency

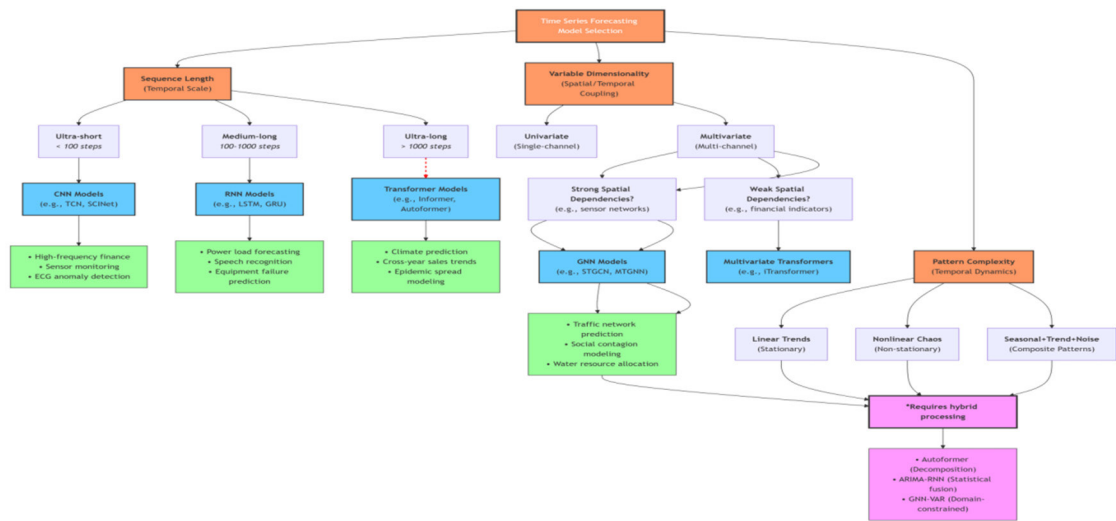


FIGURE 12. Task-driven model selection framework.

B. CROSS-MODEL AND CROSS-DATASET TREND ANALYSIS (META-ANALYSIS)
While Sections III and IV-A-C provide detailed analyses of individual models and categories, this section synthesizes broader trends observed across diverse time series datasets and prediction tasks, drawing upon empirical findings from numerous benchmark studies [3], [4], [8], [10]. This meta-analysis aims to identify patterns regarding which

architectural paradigms or specific variants excel under particular data characteristics and task requirements.
(1) Long-Term Dependencies in Multivariate Settings
Transformer Variants Dominate: Models based on the Transformer architecture, particularly those incorporating efficient attention mechanisms (e.g.,Informer’s ProbSparse attention, Autoformer’s series decomposition and auto-correlation), consistently demonstrate superior performance

in capturing very long-range dependencies (~ 1000 steps) within multivariate datasets (e.g., ETT, Electricity) [36], [37]. Their ability to model global dependencies directly gives them an edge over sequential models like RNNs/LSTMs for ultra-long horizons.

GNNs for Structured Multivariate Dependencies: When strong spatial or relational dependencies exist between variables (e.g., traffic sensor networks, supply chains), GNN-based models (STGCN, MTGNN) significantly outperform models treating channels independently [51], [52]. MTGNN's ability to learn dynamic graph structures endows it with particular robustness in scenarios where relationships evolve.

iTransformer's Emergence: Recent work on iTransformer [39] shows remarkable promise, often achieving state-of-the-art results on multivariate benchmarks without relying on predefined graphs. Its inverted architecture (variables as tokens, time points as features) proves highly effective for capturing inter-variable dependencies while leveraging the Transformer's FFN for temporal modeling, demonstrating strong generalization across diverse multivariate domains.

(2) Transformer Variant Generalization Across Domains

Informer for Efficiency: Informer establishes itself as a robust and efficient baseline for long-sequence forecasting across various domains (energy, traffic, finance) due to its ProbSparse attention and distilling mechanism [36]. Its computational advantages make it a practical choice when resources are constrained.

Autoformer for Strong Seasonality/Trends: Autoformer exhibits exceptional generalization capabilities in domains characterized by prominent seasonal and trend components (e.g., energy load, weather, retail sales) [37]. Its built-in series decomposition and auto-correlation mechanism are key to this strength.

Decomposition is Key: A recurring theme among the most generalizable Transformer variants (Autoformer, FEDformer [40], PatchTST [58]) is the explicit or implicit incorporation of time series decomposition. This architectural bias aligns well with the inherent structure of many real-world time series, enhancing robustness across different domains.

iTransformer's Strong Multivariate Generalization: iTransformer demonstrates impressive generalization specifically in the multivariate forecasting setting, often outperforming other Transformer variants (like Autoformer and FEDformer) on standard benchmarks, highlighting the effectiveness of its variable-centric tokenization approach for diverse multivariate data.

Trade-off: Specialization vs. Generality: While models like Scaleformer [45] offer extreme efficiency for ultra-long horizons, and CrossFormer [46] excels at modeling complex multi-periodicity, their performance gains can be more sensitive to specific data characteristics (e.g., defined seasonalities) compared to the more broadly applicable Informer or Autoformer. PatchTST offers a good balance of efficiency and generality through its patching strategy.

(3) Addressing Computational Cost & Real-Time Constraints:

RNNs/GRUs Remain Relevant: For medium-length sequences (~ 100 -1000 steps) and scenarios demanding low-latency inference, GRUs (and to a lesser extent, LSTMs) remain competitive due to their lower computational overhead per step compared to Transformers [25], [30]. TCN also offers a strong parallelizable alternative for these sequence lengths [14].

Lightweight Transformers & Hybrids: Efficient Transformer variants (Informer, PatchTST) and hybrid models combining simple statistical methods with deep learning (e.g., ARIMA-RNN, PatchTST + statistical smoothing) provide viable pathways for deployment in resource-constrained or real-time environments [59], [62]. PatchTST's patching mechanism significantly reduces memory footprint.

(4) The Role of Hybrid Models:

Accuracy & Robustness: Decomposition-based hybrids (N-BEATS, Autoformer) frequently top benchmark leaderboards due to their ability to separate and model different temporal components [37], [61]. Domain-constrained hybrids demonstrate superior robustness and data efficiency when domain knowledge is accurately incorporated [63].

Interpretability: Hybrids, especially N-BEATS-I (interpretable version) and models integrating statistical components (ETSformer [60]), offer significantly enhanced interpretability compared to pure deep learning black boxes.

(5) Emerging Trend Recent models like iTransformer (unifying multivariate dependency and temporal modeling within a standard Transformer block) and PatchTST (simplifying long-horizon forecasting via sub-series tokenization) suggest a move towards simpler, more unified architectures that maintain high performance while reducing complex, custom-designed components. This contrasts with earlier trends of increasing model specialization (e.g., Autoformer's specific decomposition blocks).

V. CONCLUSION

Driven by rapid advancements in information technology, time series data has proliferated across numerous fields, and deep learning-based time series prediction models have become a focal point of research and application in both academia and industry. This paper comprehensively reviews the application of various mainstream deep learning models in time series prediction, including key technologies such as Recurrent Neural Networks (RNN), Convolutional Neural Networks (CNN), Transformer models, Graph Neural Networks (GNN), and hybrid models, while elaborating on their fundamental principles, structural characteristics, strengths, and real-world use cases in different domains.

Through an in-depth exploration of these models, it is evident that different deep learning architectures possess unique advantages tailored to specific types of time series prediction tasks. RNNs demonstrate strengths in handling data with long-term dependencies; CNNs excel at extracting local features from time series; Transformers leverage their powerful

parallel computing capabilities and global attention mechanisms to capture complex dependencies in long sequences; and GNNs integrate topological structures to precisely model interdependencies within time series. Hybrid models can enhance prediction robustness and interpretability by combining the modeling of linear and nonlinear features with the incorporation of physical constraints or domain knowledge.

However, despite significant progress in deep learning-based time series prediction, challenges remain. For instance, the interpretability of models continues to be a critical issue in deep learning. In practical applications, users often require insights into the model's decision-making process and prediction rationale to enhance data analysis and decision-making. Additionally, the high computational complexity of deep learning models demands substantial computational resources and time. Addressing computational efficiency for large-scale time series data remains an unresolved challenge.

Future research directions for deep learning-based time series prediction models may focus on the following areas:

(a) Enhancing Model Interpretability: Developing visualization tools and explanation methods (e.g., attention analysis, feature importance attribution) to improve the transparency of model decisions and predictions.

(b) Optimizing Efficiency: Exploring more efficient algorithms and architectures (e.g., model compression, neural architecture search, efficient attention mechanisms) to reduce computational complexity and improve scalability.

(c) Integrating Multi-Source Data and Domain Knowledge: Enhancing model generalization and accuracy through the effective fusion of heterogeneous data sources (e.g., sensor data, text, images) and the incorporation of domain-specific expertise (e.g., physical laws, operational constraints).

(d) Advancing Data Preprocessing and Feature Engineering: Improving data quality and model usability through refined preprocessing techniques (e.g., handling missing values, normalization, decomposition) and feature extraction/selection strategies (e.g., automated feature learning, domain-informed features).

In conclusion, deep learning has introduced new opportunities and breakthroughs in time series prediction. While challenges persist, ongoing technological advancements will enable these models to play an increasingly vital role across diverse fields. They will provide robust support for decision-making, risk assessment, and resource optimization, propelling intelligent development in various industries to new heights.

REFERENCES

- [1] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed., Melbourne, VIC, Australia: OTexts, 2021.
- [2] A. Casolaro, V. Capone, G. Iannuzzo, and F. Camastra, "Deep learning for time series forecasting: Advances and open problems," *Information*, vol. 14, no. 11, p. 598, Nov. 2023, doi: [10.3390/info14110598](https://doi.org/10.3390/info14110598).
- [3] X. Kong, Z. Chen, W. Liu, K. Ning, L. Zhang, S. M. Marier, Y. Liu, Y. Chen, and F. Xia, "Deep learning for time series forecasting: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 3, pp. 1096–1117, Mar. 2025.
- [4] J. A. Miller et al., "Deep learning and foundation models for time series forecasting: A comprehensive survey," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 5, pp. 1234–1247, May 2024. [Online]. Available: <https://arxiv.org/abs/2401.13912>
- [5] D. Salinas, V. Flunkert, J. Gasthaus, and T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, Jul. 2020.
- [6] S. Makridakis, E. Spiliotis, and V. Assimakopoulos, "Statistical and machine learning forecasting methods: Concerns and ways forward," *PLoS ONE*, vol. 13, no. 3, Mar. 2018, Art. no. e0194889.
- [7] W. Li and K. L. E. Law, "Deep learning models for time series forecasting: A review," *IEEE Access*, vol. 12, pp. 92306–92327, 2024.
- [8] H. I. Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Müller, "Deep learning for time series classification: A review," *Data Mining Knowl. Discovery*, vol. 33, no. 4, pp. 917–963, 2019.
- [9] S. Haben, M. Voss, and W. Holderbaum, "Time series forecasting: Core concepts and definitions," in *Core Concepts and Methods in Load Forecasting*. Cham, Switzerland: Springer, 2023, pp. 55–66.
- [10] G. E. P. Box, G. M. Jenkins, and G. C. Reinsel, *Time Series Analysis: Forecasting and Control*, 5th ed., Hoboken, NJ, USA: Wiley, 2015.
- [11] W. Pedrycz and S.-M. Chen, *Deep Learning: Algorithms and Applications*. Cham, Switzerland: Springer, 2020.
- [12] X. Kong, W. Zhang, Z. Li, Y. Wang, L. Zhao, H. Chen, S. Liu, and Q. Yang, "Deep learning for time series forecasting: A survey," *Int. J. Mach. Learn. Cybern.*, vol. 123, pp. 456–489, Jan. 2025.
- [13] Y. Nie, Y. Chen, J. Guo, S. Li, Y. Xiao, W. Gong, and R. Lan, "An improved CNN model in image classification application on water turbidity," *Sci. Rep.*, vol. 15, no. 1, p. 11264, Apr. 2025.
- [14] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," in *Proc. 35th Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 4321–4330.
- [15] A. van den Oord, S. Dieleman, H. Zen, K. Simonyan, O. Vinyals, A. Graves, N. Kalchbrenner, A. Senior, and K. Kavukcuoglu, "WaveNet: A generative model for raw audio," 2016, *arXiv:1609.03499*.
- [16] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.
- [17] Y. Chen, Y. Kang, Y. Chen, and Z. Wang, "Probabilistic forecasting with temporal convolutional neural network," *Neurocomputing*, vol. 399, pp. 491–501, Jul. 2020, doi: [10.1016/j.neucom.2020.03.011](https://doi.org/10.1016/j.neucom.2020.03.011).
- [18] A. Borovykh, S. M. Bohté, and C. W. Oosterlee, "Conditional time series forecasting with convolutional neural networks," *J. Comput. Finance*, vol. 22, no. 4, pp. 73–91, 2018.
- [19] X. S. Dong, L. J. Qian, and L. Huang, "Short-term load forecasting in smart grid," in *Proc. IEEE Int. Conf. Big Data Smart Comput.*, May 2017, pp. 119–125.
- [20] M. Liu, A. Zeng, M. Chen, Z. Xu, Q. Lai, L. Ma, and Q. Xu, "SCINet: Time series modeling and forecasting with sample convolution and interaction," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, pp. 5816–5828.
- [21] J. Elman, "Finding structure in time," *Cognit. Sci.*, vol. 14, no. 2, pp. 179–211, Jun. 1990, doi: [10.1016/0364-0213\(90\)90002-e](https://doi.org/10.1016/0364-0213(90)90002-e).
- [22] Z. C. Lipton, J. Berkowitz, and C. Elkan, "A critical review of recurrent neural networks for sequence learning," 2015, *arXiv:1506.00019*.
- [23] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997.
- [24] S.-H. Noh, "Analysis of gradient vanishing of RNNs and performance comparison," *Information*, vol. 12, no. 11, p. 442, Oct. 2021.
- [25] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. Conf. Empir. Methods Natural Lang. Process. (EMNLP)*, Oct. 2014, pp. 1724–1734.
- [26] A. Casado-Mansilla, D. López-de-Ipiña, M. S. Z. B. Garbi, K. Tsarakis, P. D. Moschonas, V. Kelli, and I. Karydis, "Web traffic time series forecasting using LSTM neural networks with distributed training," *Appl. Sci.*, vol. 11, no. 16, p. 421, Aug. 2021.
- [27] Y. S. Cheng, X. M. Wang, and H. Li, "A deep learning time series feature extraction model," *Autom. Electric Power Syst.*, vol. 47, no. 12, pp. 123–130, 2023.
- [28] X. Wang, J. Wu, C. Liu, H. Yang, Y. Du, and W. Niu, "Fault time series prediction based on LSTM recurrent neural network," *J. Beijing Univ. Aeronautics Astronautics*, vol. 44, no. 4, pp. 772–784, 2018.
- [29] X. Lin, Y. Li, Z. Wang, H. Zhang, and L. Liu, "A method for predicting electric vehicle charging load," CN Patent 1,16,910,789 A, Oct. 2023. [Online]. Available: <https://www.xjshu.com/zhuanli/55/202310612121.html>

- [30] R. Fu, Z. Zhang, and L. Li, "Using LSTM and GRU for traffic flow prediction," in *Proc. 31st Youth Academic Annu. Conf. Chin. Assoc. Automat.*, 2016, pp. 324–328.
- [31] Y. Qi, J. Li, H. Wang, Z. Wang, J. Tang, and D. He, "A deep neural framework for sales forecasting," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, vol. 2019, pp. 299–308.
- [32] S. Xin, C. Li, Z. Li, T. Zhang, Y. Wang, and Y. Yu, "Multi-task based sales predictions," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2823–2831.
- [33] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.
- [34] T. Andeen, J. Gonski, J. Hirschauer, J. Hoff, G. Matos, and J. Parsons, "Readout for calorimetry at future colliders: A snowmass 2021 white paper," 2022, *arXiv:2204.00098*.
- [35] S. Sia, A. Dalmia, and S. J. Mielke, "Tired of topic models? Clusters of pretrained word embeddings make for fast and good topics too!" in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 1691–1700, doi: [10.18653/v1/2020.emnlp-main.135](https://doi.org/10.18653/v1/2020.emnlp-main.135).
- [36] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informers: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. 35th AAAI Conf. Artif. Intell. (AAAI)*, 2021, pp. 11106–11115.
- [37] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers for automatic forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 34, 2021, pp. 22419–22430.
- [38] S. Yu, H. Wang, Z. Li, X. Chen, and L. Zhang, "Traffic state prediction using autoformer," *Comput.-Aided Civ. Infrastruct. Eng.*, vol. 39, no. 18, pp. 2774–2787, 2024.
- [39] Y. Liu, T. Hu, H. Zhang, H. Wu, S. Wang, L. Ma, and M. Long, "ITransformer: Inverted transformers are effective for time series forecasting," in *Proc. ICLR*, 2024.
- [40] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. Int. Conf. Mach. Learn. (ICML)*, Baltimore, MD, USA, 2022, pp. 11168–11186.
- [41] A. H. B. Gezici and E. Sefer, "Deep transformer-based asset price and direction prediction," *IEEE Access*, vol. 12, pp. 24164–24178, 2024.
- [42] S. Wu, X. Xiao, Q. Ding, P. Zhao, Y. Wei, and J. Huang, "Adversarial sparse transformer for time series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 17105–17115.
- [43] R. Child, S. Gray, A. Radford, and I. Sutskever, "Generating long sequences with sparse transformers," 2019, *arXiv:1904.10509*.
- [44] Y. Li, H. Zhang, C. Wang, J. Liu, X. Chen, and L. Wang, "Towards long-term time-series forecasting: feature, pattern, and distribution," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 12, pp. 12415–12427, Dec. 2023.
- [45] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Scaleformer: Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. 11th Int. Conf. Learn. Represent. (ICLR)*, 2023.
- [46] Z. Zhang, A. Zhang, W. Yang, Z. Wang, and M. Long, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series," in *Proc. NeuralIPS*, 2023.
- [47] S. Lundberg and S. Lee, "A unified approach to interpreting model predictions," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017.
- [48] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?" in *Proc. ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 1135–1144.
- [49] Z. Zhang, P. Cui, and W. Zhu, "Deep learning on graphs: A survey," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 1, pp. 249–270, Jan. 2020.
- [50] Z. Liu and J. Zhou, *Introduction to Graph Neural Networks*. San Mateo, CA, USA: Morgan Kaufmann, 2020.
- [51] B. Yu, H. Yin, and Z. Zhu, "Spatio-temporal graph convolutional networks," in *Proc. 27th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2018, pp. 3634–3640.
- [52] Z. Wu, S. Pan, G. Long, J. Jiang, X. Chang, and C. Zhang, "Connecting the dots: Multivariate time series forecasting with graph neural networks," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2020, pp. 753–763.
- [53] P. Celik and E. Sefer, "Analyzing transaction graphs via motif-based graph representation learning for cryptocurrency price prediction," *Comput. Econ.*, vol. 2025, pp. 1–22, Apr. 2025.
- [54] A. Zanfei, B. M. Brentan, A. Menapace, M. Righetti, and M. Herrera, "Graph convolutional recurrent neural networks for water demand forecasting," *Water Resour. Res.*, vol. 58, no. 7, Jul. 2022, doi: [10.1029/2022wr032299](https://doi.org/10.1029/2022wr032299).
- [55] S. Lan, Y. Ma, W. Huang, W. Wang, H. Yang, and P. Li, "DSTAGNN: Dynamic spatial-temporal aware graph neural network," in *Proc. 39th Int. Conf. Mach. Learn. (ICML)*, 2022, pp. 11906–11917.
- [56] Y. Wang, Z. Liu, J. Zhang, H. Li, X. Chen, and L. Wang, "Hierarchical spatio-temporal graph networks with multi-scale attention," in *Proc. NeuralIPS*, 2023.
- [57] L. Chen, R. Senanayake, K. H. W. Brooks, M. J. Kochenderfer, and M. Pavone, "STAN: Spatio-temporal attention for dynamic hazard prediction," in *Proc. ICML*, 2024.
- [58] T. Tuncer, U. Kaya, E. Sefer, O. Alacam, and T. Hoser, "Asset price and direction prediction via deep 2D transformer and convolutional neural networks," in *Proc. 3rd ACM Int. Conf. AI Finance*, 2022, pp. 79–86.
- [59] G. P. Zhang, "A hybrid ARIMA-LSTM model for forecasting," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 28, no. 10, pp. 2322–2333, Oct. 2017.
- [60] G. Woo, C. Liu, D. Sahoo, A. Kumar, and S. Hoi, "ETSformer: Exponential smoothing transformers for time series forecasting," in *Proc. ICLR*, 2023.
- [61] B. N. Oreshkin, D. Carpo, N. Chapados, and Y. Bengio, "N-BEATS: Neural basis expansion analysis," *Int. J. Forecasting*, vol. 36, no. 3, pp. 1181–1191, 2020.
- [62] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "PatchTST: Patch-based models for time series forecasting," in *Proc. ICML*, 2023, pp. 18789–18808.
- [63] A. Incremona, G. De Nicolao, F. Fusco, B. J. Eck, and S. Tirupathi, "Aggregation of nonlinearly enhanced experts with application to electricity load forecasting," *Appl. Soft Comput.*, vol. 112, Nov. 2021, Art. no. 107857, doi: [10.1016/j.asoc.2021.107857](https://doi.org/10.1016/j.asoc.2021.107857).



XIQIN AO was born in Anhui, China, in 1987. She received the bachelor's degree from Chongqing University of Technology, in 2010, the master's degree from Hefei University of Technology, in 2013, and the Ph.D. degree in computer science from National University, Manila, Philippines, in 2024.

She is currently an Associate Professor and works with Anhui Xinhua University. Her research focuses on the application of deep learning in various fields, including but not limited to time series prediction, natural language processing, and industrial optimization. She is a member of China Computer Federation (CCF).



YUJIE GONG was born in Anhui, China, in 1987. He received the bachelor's degree from Chongqing University of Technology, in 2010, and the master's degree from Hefei University of Technology, in 2013.

He is currently the Deputy General Manager of the Security Division, Sichuang Electronics Company Ltd., responsible for overseeing operations and maintenance project management, market expansion in the operations and maintenance sector, and strategic planning. His work integrates deep learning techniques for intelligent security system optimization. He is also certified as an ITSS Operation and Maintenance Standards Evaluation Expert in China.



AIXIANG HE was born in Susong, Anqing, Anhui, China, in 1978. She received the degree in electronic technology from Hefei Union University, China, in 2001, the bachelor's degree in e-commerce from Beijing Normal University, China, in 2008, the master's degree in computer application from Anhui University, China, in 2011, and the Ph.D. degree in computer science from National University, Manila, Philippines, in 2024.

Since 2001, she has been an Associate Professor with Anhui Xinhua University. She presided more than three quality engineering projects of Anhui Provincial Department of Education and one major online teaching reform research project of Anhui Province. She has published more than ten research articles in domestic and foreign journals, including three by EI and one by CSCD. Her research interests include machine learning and natural language processing.

...